

Bug Tracker Pro - are tickets a strong enough pesticide?



- **Responsabili & Autori:** Valentin Gabriel Cărăuleanu [mailto:gabrielvalentine738@gmail.com], Alexandra Ghiorgă [mailto:alexandraghiorga88@gmail.com], Cleopatra Popescu [mailto:cleopatra.popescu02@gmail.com], Roberto-Giulio Pal [mailto:palrobertogiulio@gmail.com], Andrei Ionescu [mailto:andreiionescu750@gmail.com], Roșca Cvintilian [mailto:quintilianrosca2000@gmail.com]
- **Consultanți & Revizori:** Sorina-Anamaria Buf [mailto:sorinabuf@gmail.com], Ștefan Cocioran [mailto:stefancocioran@gmail.com], Florian-Luis Micu [mailto:miculuis1@gmail.com]
- **Data publicării BETA testing:** 21.10.2025
- **Deadline HARD BETA testing:** ~~30.11.2025~~ ~~07.12.2025~~ ~~10.12.2025~~ 14.12.2025 → 10:00
- **Data publicării oficiale:** 30.11.2025
- **Deadline HARD oficial:** 18.01.2026
- **Ultima modificare (cerință, schelet sau teste):**
 - schelet
 - patch schelet pentru compatibilitate devmind 21.11.2025
 - adăugare comentarii 'DO NOT CHANGE' în subsecțiune cod 'App.run' 28.11.2025
 - adăugare jackson-datatype-jsr310 dependency la cererea unui student 30.11.2025
 - fix pentru erori coding style în App.java 07.12.2025
 - enunț
 - clarificare conținut schelet 23.11.2025
 - update durată Testing Phase 15 → 12
 - adăugare warning cu privire la design pattern-urile din Lombok 30.11.2025
 - modificare câmp UIFeedback.uiElementId de la obligatoriu la opțional 4.12.2025
 - adăugare punctaje teste 04.01.2026
 - teste
 - modificare INPUT Teste 6, 15, 17 (timestamp-uri pentru consistență) 2.12.2025
 - modificare INPUT Teste 10, 12, 18, 19 (timestamp-uri pentru consistență) 10.12.2025
 - modificare REF Teste 9(overdueBy calculat incorect), 18 (tichetele dintr-un milestone blocat nu TREBUIE să își modifice prioritatea) 10.12.2025
- **Schelet și teste:** <https://github.com/oop-pub/schelet-tema2-2025> [<https://github.com/oop-pub/schelet-tema2-2025>]

Este obligatorie utilizarea a cel puțin **4 design pattern-uri diferite** ce vor fi menționate în README, explicând rolul acestora. Lipsa design pattern-urilor duce la următoarele depuneri:

- 3 design-pattern-uri: **-25%** din punctajul total al temei
- 2 design pattern-uri: **-50%** din punctajul total al temei
- 1 design pattern: **-75%** din punctajul total al temei
- 0 design pattern-uri: **-100%** din punctajul total al temei

De asemenea, dacă aveți 100p pe checker, însă soluția voastră nu este OOP, atunci veți obține o **depunere majoră** conform baremului public!

Pentru detalii consultați acest [ghid](#) care conține **baremul de corectare** și alte **guidelines**.

Atenție! Puteți folosi design pattern-urile din biblioteca **Lombok**, însă acestea nu vor fi luate în considerare pentru cele **4 design pattern-uri obligatorii**. Scopul acestui assignment este să înțelegeți design pattern-urile în întregime (structură, interacțiuni, utilizare).

- Regulamentul pentru perioada de dezvoltare BETA îl găsiți [aici](#).
- Informațiile de tip **[Nice to know]** din laborator vă pot ajuta la depanare sau la rezolvarea temei mai eficient.
- Tema a fost concepută pentru a fi rezolvată folosind **laboratoarele 1-9**, dar puteți folosi concepte și din laboratoarele următoare.
- Pentru perioada BETA, puteți folosi **laboratoarele din anii trecuți** pentru a găsi diverse informații, dar vă recomandăm să aprofundați **doar** următoarele concepte pentru această temă:
 - Sintaxa Java.
 - Clase și obiecte.
 - Moștenire și polimorfism.
 - Interfețe și clase abstracte.
 - Colecții.
 - Clase interne.
 - Excepții.
 - Static.
 - Imutabilitate.
 - File I/O.
 - Genericitate.

- Design Patterns.

Punctaje

Test	Puncte
Test Style	10.0
Test 01 - Report	2.0
Test 02 - Milestone	2.0
Test 03 - MilestoneEdgeCase	2.0
Test 04 - Assign	2.0
Test 05 - AssignEdgeCase	6.0
Test 06 - Comment	2.0
Test 07 - CommentEdgeCase	3.0
Test 08 - StatusChange	3.0
Test 09 - StatusUndoChange	3.0
Test 10 - StatusEdgeCase	3.5
Test 11 - Search	6.0
Test 12 - Notifications	6.5
Test 13 - MetricsCustomerImpact	3.5
Test 14 - MetricsTicketRisk	3.5
Test 15 - MetricsEfficiency	3.5
Test 16 - Stability	3.5
Test 17 - Performance	5.0
Test 18 - Complex	8.0
Test 19 - ComplexEdgeCase	12.0

Obiective

- Familiarizarea cu Java și conceptele de bază ale POO.
- Aprofundarea conceptelor propuse pentru tema 1.
- Folosirea claselor speciale și a bibliotecii standard Java.
- Respectarea unui stil de codare și de comentare.
- Respectarea principiilor SOLID, DRY, KISS.
- Dezvoltarea aptitudinilor de depanare a problemelor în cod.
- Implementarea a cel puțin **4 design pattern-uri diferite**.
- Familiarizarea cu Maven și a framework-urilor de testare.

Descriere

După ce ați încercat, fără succes, să obțineți un job la startup-ul lui Gigelino, ați decis să vă lansați propria aplicație. Întâmplarea a făcut să atrageți atenția unor investitori dispuși să vă finanțeze, însă cu o condiție clară: **produsul final trebuie să fie stabil, lipsit de bug-uri majore și să primească feedback pozitiv de la utilizatorii beta**.

Pentru a răspunde acestei provocări, veți implementa un **sistem de ticket tracking** care să vă ajute să:

- înregistrați tichete noi.
- raportați problemele apărute.
- urmăriți progresul și să le rezolvați.

Dezvoltarea va fi împărțită în două etape principale:

1. **testare și raportare** tichete.
2. **rezolvare** acestora de către echipa de ingineri.
3. **verificarea** aplicației și a progresului de către manageri.

Obiectivul vostru este să asigurați o gestionare eficientă a bug-urilor, prioritizarea corectă a sarcinilor și o monitorizare constantă a progresului, astfel încât să prezentați investitorilor un produs final solid și de încredere.

Atenție! Timpul este limitat, iar investitorii nu sunt dispuși să aștepte la nesfârșit.

Workflow

Dezvoltarea aplicației urmează un ciclu iterativ, alcătuit din următoarele etape:

1. **Perioada de testare:** reporterii deschid tichete pentru problemele identificate.
2. **Perioada de dezvoltare:** managerii creează milestone-uri, iar developerii preiau tichete și le rezolvă.
3. **Verificarea stabilității aplicației:** se generează un raport de stabilitate și se evaluează progresul.
 - a. Dacă produsul este stabil, proiectul se încheie cu succes.
 - b. Dacă nu, ciclul este reluat.

Acest workflow se repetă până când produsul atinge un nivel acceptabil de stabilitate **sau** investitorii decid să își retragă finanțarea **sau** se atinge deadline-ul final.

Etapele Workflow-ului

1. Perioada de Testare

- La inițierea aplicației, perioada de testare începe **automat**.
- Utilizatorii pot raporta tichete noi, pe baza testării produsului și a feedback-ului primit de la clienți.
- Developerii **nu pot** rezolva tichete în această etapă, iar managerii **nu pot** crea milestone-uri.
- Perioada de testare are o durată fixă de **12** zile.
- O nouă perioadă de testare poate fi inițiată dacă există milestone-uri active.

2. Perioada de Dezvoltare

- După încheierea perioadei de testare, începe perioada de dezvoltare.
- Managerii creează milestone-uri și își coordonează echipa.
- Developerii își pot asocia tichete din milestone-uri și pot începe lucrul la acestea.

3. Verificarea Aplicației

- Managerii pot genera rapoarte pentru monitorizarea progresului.
- Dacă raportul de stabilitate este satisfăcător, produsul este considerat **stabil** și pregătit pentru prezentarea către investitori.
- Dacă raportul indică probleme, ciclul continuă cu o nouă perioadă de testare și dezvoltare.
- Această etapă poate fi **skipped**, trecându-se direct la perioada de testare, dacă un manager execută comanda: **startTestingPhase**

Tichete

Aplicația gestionează mai multe tipuri de tichete. Toate tipurile împărtășesc un set comun de **atribute**.

Atribute comune

Câmp	Descriere	Tip	Opțional
id	Identificator unic al tichetului	int	false
type	Tipul tichetului	string	false
title	Titlu scurt	string	false
businessPriority	Nivel de prioritate în cadrul aplicației	LOW, MEDIUM, HIGH, CRITICAL	false
status	Statusul curent al tichetului	OPEN, IN_PROGRESS, RESOLVED, CLOSED	false
expertiseArea	Domeniul de expertiză necesar	FRONTEND, BACKEND, DEVOPS, DESIGN, DB	false
description	Descriere detaliată a problemei sau cererii	string	true
reportedBy	Persoana care a inițiat ticket-ul	string	

1. BUG

Un tichet de tip **BUG** reprezintă o problemă tehnică apărută în aplicație.

Atribute specifice

Câmp	Descriere	Tip	Opțional
expectedBehavior	Comportamentul așteptat	string	false
actualBehavior	Comportamentul observat	string	false
frequency	Frecvența apariției	RARE, OCCASIONAL, FREQUENT, ALWAYS	false
severity	Severitatea bug-ului (cât de mult afectează aplicația)	MINOR, MODERATE, SEVERE	false
environment	Mediu de operare	String	true
errorCode	Cod de eroare returnat de aplicație	int	true

Atenție! Doar un tichet de tip **BUG** poate fi raportat **ANONIM**.

2. FEATURE_REQUEST

Un tichet de tip **FEATURE_REQUEST** propune o nouă funcționalitate sau extinderea uneia existente.

Atribute specifice

Câmp	Descriere	Valori posibile	Opțional
businessValue	Impact estimat asupra afacerii	S, M, L, XL	false
customerDemand	Cererea clienților pentru această funcționalitate	LOW, MEDIUM, HIGH, VERY_HIGH	false

3. UI_FEEDBACK

Un tichet de tip **UI_FEEDBACK** colectează sugestii privind experiența de utilizare.

Atribute specifice

Câmp	Descriere	Tip	Opțional
uiElementId	Identificatorul elementului de UI	string	true
businessValue	Impact estimat asupra utilizatorilor sau business-ului	S, M, L, XL	false
usabilityScore	Cât de acceptabilă este versiunea curentă	int(1-10)	false
screenshotUrl	Link către o captură de ecran	string	true

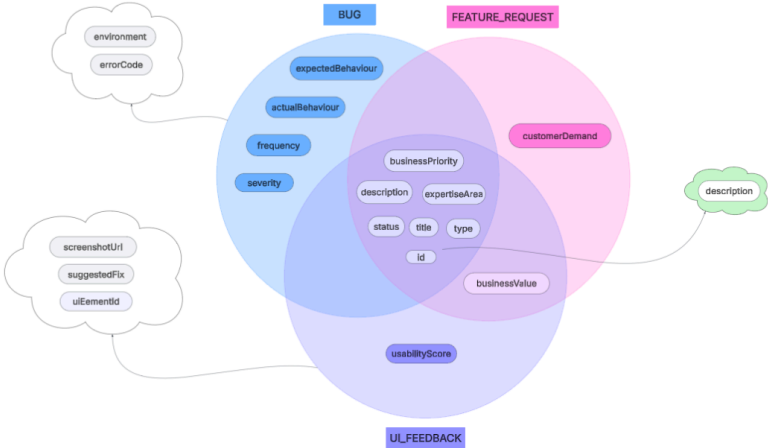
suggestedFix	Sugestie de îmbunătățire pentru UI	string	true
--------------	------------------------------------	--------	------

Tip! Fiecare tip de tichet definit în aplicație conține atât atribute **obligatorii**, cât și atribute **opționale**. De aceea, instanțierea și construirea obiectelor trebuie gândită și proiectată cu atenție.

Câmpurile prezentate anterior sunt cele obligatorii, primite în comenzile **reportTicket** și **createMilestone**, însă puteți adăuga orice câmp considerați necesar.

Rezumat

O imagine valoreaza o mie de cuvinte.



Utilizatori

Aplicația definește trei tipuri de utilizatori, fiecare cu roluri și atribute specifice. Datele de intrare pentru utilizatori se află în fișierul-schelet: **input/database/users.json**.

Un utilizator este definit prin:

Câmp	Descriere	Valori posibile	Optional
username	Numele de utilizator al persoanei.	string	false
email	Adresa de e-mail asociată utilizatorului.	string	false
role	Rolul utilizatorului în sistem.	REPORTER, DEVELOPER, MANAGER	false

Reporter

Reporterul are rolul de a raporta tichete în perioadele de testare pe baza feedback-ului primit de la clienți și a testării interne. De asemenea, poate primi notificări legate de statusul tichetelor raportate.

```
[
  {
    "username": "reporter_one",
    "role": "REPORTER",
    "email": "reporter_one@gmail.com"
  },
  {
    "username": "reporter_two",
    "role": "REPORTER",
    "email": "reporter_two@gmail.com"
  }
]
```

Developer

Developerul se ocupă de rezolvarea tichetelor raportate și poate primi notificări legate de activitatea acestora.

Un developer este definit prin:

Câmp	Descriere	Valori posibile	Optional
hireDate	Data angajării dezvoltatorului.	"yyyy-mm-dd" (string)	false
expertiseArea	Domeniile de expertiză ale dezvoltatorului.	FRONTEND, BACKEND, DEVOPS, DESIGN, DB, FULLSTACK	false
seniority	Nivelul de experiență al dezvoltatorului.	JUNIOR, MID, SENIOR	false

Not all devs are made equal. De aceea, în funcție de experiența profesională, există 3 niveluri de senioritate care determină accesul acestora la tichete.

Acces la tichete în funcție de senioritate:

Seniority	Acces Prioritate	Tipuri Tichete
-----------	------------------	----------------

JUNIOR	LOW, MEDIUM	BUG, UI_FEEDBACK
MID	LOW, MEDIUM, HIGH	BUG, UI_FEEDBACK, FEATURE_REQUEST
SENIOR	LOW, MEDIUM, HIGH, CRITICAL	BUG, UI_FEEDBACK, FEATURE_REQUEST

Mai mult! Fiecare developer are o specializare:

Specializare	Zone accesibile
FRONTEND	FRONTEND, DESIGN
BACKEND	BACKEND, DB
FULLSTACK	FRONTEND, BACKEND, DEVOPS, DESIGN, DB
DEVOPS	DEVOPS
DESIGN	DESIGN, FRONTEND
DB	DB

```
[
  {
    "username": "dev_three",
    "role": "DEVELOPER",
    "email": "dev_three@bugtracker.com",
    "hireDate": "2022-07-22",
    "seniority": "JUNIOR",
    "expertiseArea": "FRONTEND"
  },
  {
    "username": "dev_four",
    "role": "DEVELOPER",
    "email": "dev_four@bugtracker.com",
    "hireDate": "2019-11-05",
    "seniority": "MID",
    "expertiseArea": "FULLSTACK"
  }
]
```

Tip! Toți developerii execută aceleași comenzi, dar senioritatea și expertiza determină **dacă o acțiune este permisă**. Think of **Separation of Concerns**.

Manager

Managerul coordonează propria echipa de developeri, creează și gestionează **milestone-uri** și menține o listă de subordonați reprezentată prin username-urile developerilor pe care îi are în echipă.

Câmp	Descriere	Valori posibile	Opțional
hireDate	Data angajării persoanei.	string	false
subordinates	Lista subordonaților (username-uri).	string[]	false

```
[
  {
    "username": "gabriel_manager",
    "role": "MANAGER",
    "email": "gabriel@bugtracker.com",
    "hireDate": "2002-02-28",
    "subordinates": ["dev_one", "dev_two", "dev_three"]
  },
  {
    "username": "cleopatra_manager",
    "role": "MANAGER",
    "email": "cleopatra@bugtracker.com",
    "hireDate": "2003-03-15",
    "subordinates": ["dev_four", "dev_five"]
  }
]
```

Milestones

Un **milestone** reprezintă o etapă majoră în dezvoltarea proiectului. Acesta este stabilit de un **manager** la finalul unei perioade de testare și conține un set de tichete care trebuie rezolvate într-un interval de timp predefinit.

Doar **developerii** care fac parte din **echipa managerului** respectiv pot fi atribuiți aceluși milestone.

Atributele unui milestone

Câmp	Descriere	Tip	Opțional
name	Identificatorul milestone-ului	string	false
blockingFor	Listă de nume ale celorlalte milestone-uri care sunt blocate de acest milestone	string[]	false
dueDate	Data limită pentru atingerea milestone-ului	date	false
tickets	Listă de ID-uri ale tichetelor asociate milestone-ului	int[]	false
assignedDevs	Listă cu username-urile developerilor repartizați	string[]	false

Mentiuni speciale

1. La fiecare **3 zile** după crearea milestone-ului, **prioritatea tichetelor din acel milestone crește automat cu o treaptă** (ex: **LOW** → **MEDIUM**, **MEDIUM** → **HIGH**, **CRITICAL** → **CRITICAL**).
2. Cu **o zi calendaristică** înainte de **dueDate**:

1. toate tichetele rămase active capătă prioritatea **CRITICAL**.

2. toți developerii repartizați acelui milestone primesc o **notificare**.
3. Un **milestone poate bloca alte milestone-uri**. Dacă un milestone se află în starea de **blocare**:

1. bug-urile din milestone-ul **blocat** nu pot fi rezolvate sau preluate de un developer.

2. regulile de la punctele **1** și **2** nu se aplică (prioritatea tichetelor nu crește, iar acestea nu devin **CRITICAL**).
4. Dacă un milestone este deblocat cu **o zi calendaristică** înainte de **dueDate**, regula de la punctul **2** se aplică.
5. Dacă un milestone este deblocat **după** dueDate:

1. toate tichetele rămase devin automat **CRITICAL**.

2. se trimite o **notificare specială** developerilor repartizați acelui milestone.

Atenție!

Un milestone este deblocat în momentul în care ultimul tichet din milestone-ul blocant devine **CLOSED**.

Vom defini următoarele convenții:

- **daysBetween = date2 – date1 + 1**, pentru a include ambele zile.
- **O zi calendaristică** înainte de YYYY-MM-DD înseamnă întotdeauna ziua calendaristică anterioară.
- **Exemple:** Pentru intervalul **2023-05-01** → **2023-05-10**, daysBetween = **10**. Dacă dueDate = **2023-05-10**, ziua calendaristică anterioară este **2023-05-09**

Tip! Pentru lucrul cu datele, recomandăm următorul flow:

```
LocalDate now = LocalDate.parse(nowString); // conversia dintr-un string într-un obiect LocalDate
LocalDate due = LocalDate.parse(dueString);

int daysBetween = (int) ChronoUnit.DAYS.between(now, due) + 1; // calculul daysBetween
```

Atenție!

Dacă prioritatea unui tichet ajunge să depășească senioritatea unui developer la care este repartizat, tichetul devine automat **OPEN**.

Acest caz va apărea doar în testul **complex_edge**.

Notificări

Întreaga aplicație este bazată pe un **sistem de notificări**. Orice acțiune semnificativă (ex: adăugare comentariu, schimbare status tichet, creare milestone) va genera o notificare către utilizatorii vizai.

Acțiune	Cine primește	Format notificare
Tichet trecut la CLOSED , ultimul dintr-un milestone blocant	Toți developerii atribuiți milestone-urilor blocate	Milestone <milestone_name> is now unblocked as tichet <id> has been CLOSED.
Crearea unui Milestone	Toți developerii atribuiți milestone-ului	New milestone <milestone_name> has been created with due date <due_date>.
Milestone ajuns la o zi de dueDate	Toți developerii repartizați milestone-ului	Milestone <milestone_name> is due tomorrow. All unresolved tickets are now CRITICAL.
Milestone deblocat după dueDate	Toți developerii repartizați milestone-ului	Milestone <milestone_name> was unblocked after due date. All active tickets are now CRITICAL.

Tip! Pentru a gestiona notificările într-un mod scalabil, gândiți-vă la o soluție în care utilizatorii să fie automat informați.

Metrici de Stabilitate

Pe lângă rapoartele de performanță, aplicația generează și **rapoarte** pe baza unor metrici specifice, calculate în funcție de tipul fiecărui tichet.

Cele 3 metrici cu care vom lucra sunt: **Customer Impact**, **Resolution Efficiency**, **Ticket Risk**.

```
frequency:
  RARE      = 1
  OCCASIONAL = 2
  FREQUENT  = 3
  ALWAYS    = 4

businessPriority:
  LOW       = 1
  MEDIUM   = 2
  HIGH      = 3
  CRITICAL  = 4

severityFactor:
  MINOR     = 1
  MODERATE  = 2
  SEVERE    = 3

businessValue:
  S         = 1
  M         = 3
  L         = 6
  XL        = 10

customerDemand:
  LOW       = 1
  MEDIUM   = 3
```

```
HIGH      = 6
VERY_HIGH = 10
```

Pași generali de calcul

1. Pentru fiecare tichet eligibil se calculează **scorul brut** (baseScore), în funcție de formula asociată.
2. Se normalizează rezultatul pe intervalul **0–100**.
3. Se calculează media scorurilor pentru o categorie.

```
public double calculateImpactFinal(double baseScore, double maxValue) {
    return Math.min(100.0, (baseScore * 100.0) / maxValue);
}
```

```
public double calculateAverageImpact(List<Double> scores) {
    return scores.stream().mapToDouble(Double::doubleValue).average().orElse(0.0);
}
```

Tip! Structura codului ar trebui să permită adăugarea unor metrice noi, fără a altera clasele deja existente.

Code should be open for extension, closed for modification.

Mai multe detalii, veți găsi la comenzile specifice fiecărei metrici.

Comenzi

Aplicația este **user-centric**, ceea ce înseamnă că fiecare comandă primită este asociată unui **utilizator** identificat prin câmpul **username**.

De exemplu, dacă un utilizator trimite comanda **changeStatus**, aceasta va modifica starea unui ticket în funcție de acțiunea solicitată. Comanda **undoChangeStatus** va anula ultima modificare de stare efectuată de **acel** utilizator.

Error Handling

Pe parcursul execuției, va trebui să gestionați diverse erori.

Ordinea în care apar în enunț este și ordinea în care trebuie verificate. Astfel, **excepția menționată prima** va avea întotdeauna **prioritate de tratare** față de cele care apar mai jos. Următoarele excepții pot apărea la orice comandă.

Utilizator inexistent

Dacă un utilizator trimite o comandă, dar **nu există în sistem**, comanda este invalidă și trebuie afișată o eroare corespunzătoare.

```
{
  "command": "command_name",
  "username": "user_name",
  "timestamp": "YYYY-MM-DD",
  "error": "The user <user_name> does not exist."
}
```

Comandă nepermisă pentru rol

Dacă utilizatorul nu are rolul necesar pentru a rula o comandă, aceasta este refuzată cu mesaj de eroare clar.

```
{
  "command": "command_name",
  "username": "user_name",
  "timestamp": "YYYY-MM-DD",
  "error": "The user does not have permission to execute this command: required role <role_name1>, <role_name2>; user role <user_role>."
}
```

Rolurile vor fi afișate în ordinea apariției lor.

Exemplu: Comanda X este **disponibilă pentru MANAGER, DEVELOPER**, iar un reporter încearcă să o acceseze. **Error** va fi :

The user does not have permission to execute this command: required role MANAGER, DEVELOPER; user role REPORTER.

Aceste excepții sunt verificate **înainte** de orice validări specifice fiecărei comenzi.

Atenție! Toate rezultatele numerice vor de tip **Double** cu 2 zecimale și rotunjite folosind **Math.round(n * 100.0) / 100.0**

Acum putem trece la comenzile aplicației.

Flow Management

Pierderea investitorilor

Disponibilă pentru: Manager

Descriere: Începem cu pesimism. Investitorii și-au pierdut răbdarea și au renunțat la finanțare. Aplicația se închide și **nu mai sunt procesate comenzi** după acest moment.

Comanda: lostInvestors

```
{
  "command": "lostInvestors",
  "username": "gabriel_manager",
  "timestamp": "2025-10-20"
}
```

Nu există output.

Începerea unei noi perioade de testare

Disponibilă pentru: Manager

Descriere: Începe o nouă perioadă de testare, permițând reporterilor să creeze tichete noi.

Command: startTestingPhase

Restricții

O nouă perioadă de testare poate începe doar dacă nu mai există milestone-uri active (cu tichete nerezolvate).

În cazul în care se încearcă începerea unei noi perioade de testare cu milestone-uri active, comanda este respinsă:

```
{
  "command": "startTestingPhase",
  "username": "user_name",
  "timestamp": "YYYY-MM-DD",
  "error": "Cannot start a new testing phase."
}
```

Se garantează că nu se va încerca începerea unei perioade de testare în timpul altei perioade de testare .

Exemplu Input:

```
{
  "command": "startTestingPhase",
  "username": "gabriel_manager",
  "timestamp": "2025-10-05"
}
```

Nu există output.

Ticket Management

Crearea tichetelor

Disponibilă pentru: Reporter

Descriere: Reportarea este permisă doar în perioada de testare. Pentru mai multe detalii, consultați secțiunea [Perioada de Testare](#).

Comanda: reportTicket

Restricții:

1. Tichetele pot fi raportate doar în perioada de testare.

```
{
  "command": "reportTicket",
  "username": "user_name",
  "timestamp": "YYYY-MM-DD",
  "error": "Tickets can only be reported during testing phases."
}
```

2. Raportarea anonimă:

- Tichetele anonime au automat **LOW** businessPriority.
- Doar tichetele de tip **BUG** pot fi anonime (**reportedBy** = "").


```
{
  "command": "reportTicket",
  "username": "user_name",
  "timestamp": "YYYY-MM-DD",
  "error": "Anonymous reports are only allowed for tickets of type BUG."
}
```

ID-urile sunt alocate incremental de sistem, începând de la **0**.

Se garantează că formatul de input este corect (fără câmpuri invalide sau lipsă).

Statusul inițial al unui tichet este întotdeauna **OPEN**.

Atributul **username** și **reportedBy** nu trebuie să aibă aceeași valoare, dar notificările ulterioare sunt primite de către reporter-ul oficial (definit prin **username**).

Format Input:

```
{
  "command": "reportTicket",
  "timestamp": "2025-09-10",
  "username": "reporter_one",
  "params": {
    "type": "BUG",
    "title": "Crash on settings page",
    "description": "App crashes when accessing settings.",
    "businessPriority": "HIGH",
    "frequency": "FREQUENT",
    "expertiseArea": "BACKEND",
    "reportedBy": "cleopatra",
    "expectedBehavior": "Settings page should load without crashing.",
    "actualBehavior": "App crashes immediately.",
    "environment": "Windows"
  }
}
```

Nu există output.

Vizualizarea tichetelor

Disponibilă pentru: **Developer, Manager, Reporter**

Descriere: Afișează tichetele din sistem în funcție de rolul utilizatorului curent.

Comanda: **viewTickets**

Restricții

1. **Developer:** vede toate tichetele cu status **OPEN** din milestone-urile la care este repartizat (indiferent dacă le poate prelua spre soluționare).
2. **Manager:** vede toate tichetele din sistem.
3. **Reporter:** vede doar tichetele introduse de acesta în sistem. (tichetele anonime sunt excluse).

Rezultatele sunt sortate după **createdAt** (de la cele mai vechi la cele mai noi), iar în caz de egalitate după **ID** crescător.

solvedAt este timestamp-ul la care un tichet a ajuns în statusul **RESOLVED**.

Exemplu Input:

```
{
  "command": "viewTickets",
  "username": "manager_one",
  "timestamp": "2025-09-25"
}
```

Exemplu Output:

```
{
  "command": "viewTickets",
  "username": "dev_one",
  "timestamp": "2025-09-25",
  "tickets": [
    {
      "id": 0,
      "type": "BUG",
      "title": "Crash on settings page",
      "businessPriority": "HIGH",
      "status": "IN_PROGRESS",
      "createdAt": "2025-09-10",
      "solvedAt": "",
      "assignedAt": "2025-09-21",
      "assignedTo": "dev_one",
      "reportedBy": "cleopatra",
    }
  ]
}
```

```
"comments": [  
  {  
    "author": "dev_one",  
    "content": "I am looking into this issue.",  
    "createdAt": "2025-09-10"  
  },  
  {  
    "author": "manager_gabriel",  
    "content": "Please prioritize this bug.",  
    "createdAt": "2025-09-11"  
  }  
]  
]  
]  
}
```

Adăugarea de comentarii

Disponibilă pentru: Reporter, Developer

Descriere: Adaugă un comentariu la un tichet.

Comanda: addComment

Restricții

1. Dacă tichetul nu există (ID inexistent), comanda este ignorată.
2. Un tichet anonim nu poate primi comentarii. Dacă se încearcă adăugarea unui comentariu la un tichet anonim, comanda este respinsă:

```
{  
  "command": "addComment",  
  "username": "user_name",  
  "timestamp": "YYYY-MM-DD",  
  "error": "Comments are not allowed on anonymous tickets."  
}
```

3. Un **Reporter** nu poate comenta dacă tichetul are status **CLOSED**. În cazul în care se încearcă comentarea unui tichet cu status **CLOSED**, comanda este respinsă:

```
{  
  "command": "addComment",  
  "username": "user_name",  
  "timestamp": "YYYY-MM-DD",  
  "error": "Reporters cannot comment on CLOSED tickets."  
}
```

4. Conținutul comentariului trebuie să aibă minim **10 caractere**. Dacă este mai mic, comanda este respinsă:

```
{  
  "command": "addComment",  
  "username": "user_name",  
  "timestamp": "YYYY-MM-DD",  
  "error": "Comment must be at least 10 characters long."  
}
```

5. Un **Developer** poate comenta doar pe tichetele pe care și le-a repartizat pentru soluționare. În caz contrar, comanda este respinsă:

```
{  
  "command": "addComment",  
  "username": "user_name",  
  "timestamp": "YYYY-MM-DD",  
  "error": "Ticket <id> is not assigned to the developer <developer_name>."  
}
```

6. Un **Reporter** poate comenta doar pe tichetele pe care le-a raportat. În caz contrar, comanda este respinsă:

```
{  
  "command": "addComment",  
  "username": "user_name",  
  "timestamp": "YYYY-MM-DD",  
  "error": "Reporter <reporter_name> cannot comment on ticket <id>."  
}
```

Format comentariu

```
{
  "author": "username_of_commenter",
  "content": "string",
  "createdAt": "date_given_by_command_timestamp"
}
```

Exemplu Input:

```
{
  "command": "addComment",
  "username": "dev_one",
  "ticketID": 0,
  "comment": "I am looking into this issue.",
  "timestamp": "2025-09-10"
}
```

Nu există output.

Ștergerea de comentarii

Disponibilă pentru: Reporter, Developer

Descriere: Șterge ultimul comentariu adăugat de utilizator la un tichet.

Comanda: `undoAddComment`

Restricții

1. Dacă tichetul nu există (ID inexistent) sau dacă utilizatorul nu are comentarii la tichetul respectiv comanda este ignorată.
2. Dacă se încearcă ștergerea unui comentariu de la un tichet anonim, comanda este respinsă:

```
{
  "command": "undoAddComment",
  "username": "user_name",
  "timestamp": "YYYY-MM-DD",
  "error": "Comments are not allowed on anonymous tickets."
}
```

Exemplu Input:

```
{
  "command": "undoAddComment",
  "username": "dev_one",
  "ticketID": 0,
  "timestamp": "2025-09-10"
}
```

Nu există output.

Milestone Management

Crearea unui milestone

Disponibilă pentru: Manager

Descriere: Setează un nou milestone.

Comanda: `createMilestone`

Restricții

- Un tichet poate aparține unui singur milestone la un moment dat. În cazul în care un tichet este deja atribuit unui alt milestone, comanda **createMilestone** este respinsă:

```
{
  "command": "createMilestone",
  "username": "user_name",
  "timestamp": "YYYY-MM-DD",
  "error": "Tickets id already assigned to milestone <milestone_name>."
}
```

Dacă se încearcă crearea unui milestone cu mai multe tichete din alt milestone, comanda va eșua la primul tichet invalid.

Se garantează că formatul de input al milestone-ului este corect. (Fără câmpuri invalide sau lipsă câmpuri obligatorii). Nu vor exista milestone-uri cu același nume.

Se garantează că orice milestone va avea o perioadă de soluționare de cel puțin **5 zile**.

Se garantează că nu se va încerca crearea unui milestone în perioada de testare.

Exemplu Input:

```
{
  "command": "createMilestone",
  "username": "gabriel_manager",
  "timestamp": "2025-09-15",
  "name": "v1.0",
  "dueDate": "2025-10-01",
  "blockingFor": ["v2.0"],
  "tickets": [0, 1, 2],
  "assignedDevs": ["dev_one", "dev_two"]
}
```

Nu există output.

Vizualizarea milestone-urilor

Disponibilă pentru: **Manager, Developer**

Descriere: Afișează milestone-urile din sistem în funcție de rolul utilizatorului curent.

Comanda: `viewMilestones`

Restricții

1. **Manager:** vede toate milestone-urile create de el.
2. **Developer:** vede toate milestone-urile la care este repartizat.

Milestone-urile sunt afișate crescător după **dueDate**, iar în caz de egalitate se va ordona lexicografic crescător după **name**.

Când un milestone devine inactiv (ultimul tichet devine CLOSED), **overdueBy** și **daysUntilDue** rămân blocate.

Exemplu: un milestone are **dueDate** pe 20 oct. Pe 18 oct ultimul tichet este închis. Comanda **viewMilestones** de pe 21 octombrie va afișa **daysUntilDue = 3** și **overdueBy = 0**.

Calculat ca raportul dintre numărul de tichete **CLOSED** și numărul total de tichete din milestone. Valoare de tip **Double**, afișată cu **2 zecimale**.

Numărul de zile rămase până la **dueDate**. Dacă **dueDate** este în trecut, valoarea este **0**. **Formulă:** $\text{dueDate} - \text{currentDay} + 1$

Numărul de zile cu care **dueDate** a fost depășit. Dacă **dueDate** este în viitor, valoarea este **0**.

Listă de obiecte ce conțin:

- numele developerilor responsabili pentru milestone
- ID-urile tichetelor atribuite fiecăruia

Lista este:

- sortată **crescător** după numărul de tichete atribuite
- în caz de egalitate, după **username** (ordine alfabetică).

Poate avea două valori:

- **ACTIVE** – dacă există tichete nerezolvate în milestone
- **COMPLETED** – dacă **toate** tichetele sunt **CLOSED**

Exemplu Input:

```
{
  "command": "viewMilestones",
  "username": "username",
}
```

```
{
  "timestamp": "2025-09-25"
}
```

Exemplu Output:

```
{
  "command": "viewMilestones",
  "username": "pm_lead",
  "timestamp": "2025-09-25",
  "milestones": [
    {
      "name": "v1.0",
      "blockingFor": ["v2.0"],
      "dueDate": "2025-10-01",
      "createdAt": "2025-09-15",
      "tickets": [0, 1, 2],
      "assignedDevs": ["dev_one", "dev_two"],
      "createdBy": "gabriel_manager",
      "status": "ACTIVE",
      "isBlocked": false,
      "daysUntilDue": 6,
      "overdueBy": 0,
      "openTickets": [0, 1, 2],
      "closedTickets": [],
      "completionPercentage": 0.00,
      "repartition": [
        {
          "developer": "dev_one",
          "assignedTickets": [1]
        },
        {
          "developer": "dev_two",
          "assignedTickets": [2]
        },
        {
          "developer": "dev_three",
          "assignedTickets": []
        }
      ]
    }
  ]
}
```

Ticket Development

Preluarea unui tichet

Disponibilă pentru: Developer

Descriere: Un developer își poate atribui un tichet conform regulilor de expertiză, senioritate și milestone.

Comanda: assignTicket

Restricții

1. Un **Developer** poate prelua tichete doar din aria lui de expertiză. În caz contrar, comanda este respinsă:

```
{
  "command": "assignTicket",
  "username": "user_name",
  "timestamp": "YYYY-MM-DD",
  "error": "Developer <developer_name> cannot assign ticket <id> due to expertise area. Required: <required_area1>, <required_area2>; Current: <current_area>."
}
```

Atenție!

required_areas sunt sortate lexicografic.

2. Un **Developer** poate prelua singur doar tichete accesibile nivelului său de senioritate. În caz contrar, comanda este respinsă:

```
{
  "command": "assignTicket",
  "username": "user_name",
  "timestamp": "YYYY-MM-DD",
  "error": "Developer <developer_name> cannot assign ticket <id> due to seniority level. Required: <required_level>, <required_level>; Current: <current_level>."
}
```

Atenție!

required_levels sunt sortate lexicografic.

3. Un tichet poate fi preluat doar dacă are statusul **OPEN**. În caz contrar, comanda este respinsă:

```
{
  "command": "assignTicket",
  "username": "user_name",
  "timestamp": "YYYY-MM-DD",
  "error": "Only OPEN tickets can be assigned."
}
```

4. Dacă un developer încearcă să preia un tichet dintr-un milestone la care nu este repartizat, comanda este respinsă:

```
{
  "command": "assignTicket",
  "username": "user_name",
  "timestamp": "YYYY-MM-DD",
  "error": "Developer <developer_name> is not assigned to milestone <milestone_name>."
}
```

5. Un developer nu poate prelua tichete dintr-un milestone **blocat**. În caz contrar, comanda este respinsă:

```
{
  "command": "assignTicket",
  "username": "user_name",
  "timestamp": "YYYY-MM-DD",
  "error": "Cannot assign ticket <id> from blocked milestone <milestone_name>."
}
```

Odată atribuit, statusul unui tichet devine **IN_PROGRESS**.

Se garantează că ID-ul tichetului există.

Exemplu Input:

```
{
  "command": "assignTicket",
  "username": "dev_one",
  "ticketID": 0,
  "timestamp": "2025-09-21"
}
```

Nu există output.

Renuntarea la un tichet

Disponibilă pentru: Developer

Descriere: Developer-ul realizează că nu poate rezolva un tichet și decide să renunțe, pentru a fi preluat de altcineva.

Comanda: `undoAssignTicket`

Restricții

1. Un tichet poate fi „de-asignat” doar dacă are statusul **IN_PROGRESS**. În cazul în care un developer încearcă să renunțe la un tichet cu alt status, comanda este respinsă:

```
{
  "command": "undoAssign",
  "username": "user_name",
  "timestamp": "YYYY-MM-DD",
  "error": "Only IN_PROGRESS tickets can be unassigned."
}
```

Se garantează că ID-ul tichet-ului pentru care se încearcă renunțarea există.

În urma renunțării la un tichet, statusul acestuia devine **OPEN**.

Operația rămâne în istoric. Pentru detalii suplimentare, vezi comanda **viewTicketHistory**.

Exemplu Input:

```
{
  "command": "undoAssign",
  "username": "dev_one",
  "ticketID": 0,
  "timestamp": "2025-09-22"
}
```

Nu există output.

Vizualizarea tichetelor repartizate unui developer

Disponibilă pentru: Developer

Descriere: Afișează toate tichetele repartizate unui developer.

Comanda: viewAssignedTickets

Afișarea este realizată după **businessPriority** (CRITICAL > HIGH > MEDIUM > LOW), apoi după **createdAt** crescător, apoi după **id** crescător.

Exemplu Input:

```
{
  "command": "viewAssignedTickets",
  "username": "dev_one",
  "timestamp": "2025-09-25"
}
```

Exemplu Output:

```
{
  "command": "viewAssignedTickets",
  "username": "dev_one",
  "timestamp": "2025-09-25",
  "assignedTickets": [
    {
      "id": 0,
      "type": "BUG",
      "title": "Crash on settings page",
      "businessPriority": "HIGH",
      "status": "IN_PROGRESS",
      "createdAt": "2025-09-10",
      "assignedAt": "",
      "reportedBy": "cleopatra",
      "comments": [
        {
          "author": "dev_one",
          "content": "I am looking into this issue.",
          "createdAt": "2025-09-10"
        },
        {
          "author": "reporter_one",
          "content": "Please prioritize this bug.",
          "createdAt": "2025-09-11"
        }
      ]
    },
    {
      "id": 2,
      "type": "FEATURE_REQUEST",
      "title": "Add dark mode",
      "businessPriority": "MEDIUM",
      "status": "RESOLVED",
      "createdAt": "2025-09-12",
      "assignedAt": "",
      "reportedBy": "user123",
      "comments": []
    }
  ]
}
```

Schimbarea statusului unui tichet

Disponibilă pentru: Developer

Descriere: Permite unui developer să modifice statusul unui tichet preluat.

Comenzi: changeStatus, undoChangeStatus

Restricții

1. Tichetul trebuie să fie repartizat developerului care execută comanda. În caz contrar, comanda este respinsă:

```
{
  "command": "changeStatus",
  "username": "user_name",
  "timestamp": "YYYY-MM-DD",
  "error": "Ticket <id> is not assigned to developer <developer_name>."
}
```

2. Dacă tichetul are statusul **IN_PROGRESS**, comanda **undoChangeStatus** este ignorată.

3. Dacă tichetul are statusul **CLOSED**, comanda **changeStatus** este ignorată.

Cele 2 comenzi au input identic, diferența fiind atributul **command**.

Dacă un tichet este redeschis dintr-un milestone anterior blocant, milestone-urile deblocate rămân deblocate.

Exemplu Input changeStatus/undoChangeStatus:

```
{
  "command": "changeStatus",
  "username": "dev_one",
  "ticketID": 0,
  "timestamp": "2025-09-23"
}
```

Nu există output.

Vizualizarea istoricului tichetelor

Disponibilă pentru: Developer, Manager

Descriere: Un user vede istoricul tichetelor.

Comanda: viewTicketHistory

Restricții

- Un **Developer** poate vedea istoricul tichetelor care i-au fost repartizate (inclusiv cele **CLOSED**) și pe cele la care a renunțat.
- Un **Manager** poate vedea istoricul tichetelor care au fost în milestone-urile pe care le-a creat, pentru toți developerii implicați.

Istoricul unui tichet la care un developer a renunțat nu conține și informații ulterioare momentului comenzii **undoAssignTicket**.

Rezultatele sunt sortate după data **createdAt**, cele mai vechi fiind afișate primele, iar în caz de egalitate după **id**.

Acțiunile apar în ordinea în care au fost efectuate.

Acțiuni posibile

Descriere: tichet-ul a fost atribuit unui developer.

Exemplu JSON:

```
{
  "action": "ASSIGNED",
  "by": "developer_name",
  "timestamp": "YYYY-MM-DD"
}
```

Descriere: developer-ul a renunțat la tichet.

Exemplu JSON:

```
{
  "action": "DE-ASSIGNED",
  "by": "developer_name",
  "timestamp": "YYYY-MM-DD"
}
```

Descriere: status-ul tichetului a fost schimbat.

Exemplu JSON:

```
{
  "action": "STATUS_CHANGED",
  "from": "OLD_STATUS",
  "to": "NEW_STATUS",
  "by": "developer_name",
  "timestamp": "YYYY-MM-DD"
}
```

Descriere: tichetul a fost adăugat într-un milestone.

Exemplu JSON:

```
{
  "action": "ADDED_TO_MILESTONE",
  "milestone": "milestone_name",
  "by": "manager_name",
}
```



```
"timestamp": "YYYY-MM-DD"
}
```

Descriere: tichetul devine **OPEN** pentru că depășește prioritatea admisă nivelul de senioritatea al dev-ului.

Exemplu JSON:

```
{
  "action": "REMOVED_FROM_DEV",
  "by": "system",
  "from": "developer_name",
  "timestamp": "YYYY-MM-DD"
}
```

Exemplu Input:

```
{
  "command": "viewTicketHistory",
  "username": "dev_one",
  "timestamp": "2025-09-25"
}
```

Exemplu Output:

```
{
  "command": "viewTicketHistory",
  "username": "dev_one",
  "timestamp": "2025-09-25",
  "ticketHistory": [
    {
      "id": 0,
      "title": "Ticket 1",
      "status": "CLOSED",
      "actions": [
        {
          "action": "ASSIGNED",
          "by": "dev_one",
          "timestamp": "2025-09-15"
        },
        {
          "action": "STATUS_CHANGED",
          "from": "OPEN",
          "to": "IN_PROGRESS",
          "by": "dev_one",
          "timestamp": "2025-09-15"
        },
        {
          "action": "STATUS_CHANGED",
          "from": "IN_PROGRESS",
          "to": "RESOLVED",
          "by": "dev_one",
          "timestamp": "2025-09-18"
        },
        {
          "action": "STATUS_CHANGED",
          "from": "RESOLVED",
          "to": "CLOSED",
          "by": "dev_one",
          "timestamp": "2025-09-20"
        }
      ],
      "comments": [
        {
          "author": "dev_one",
          "timestamp": "2025-09-20",
          "message": "Fixed the issue."
        }
      ]
    },
    {
      "id": 1,
      "title": "Ticket 2",
      "status": "IN_PROGRESS",
      "actions": [
        {
          "action": "ASSIGNED",
          "by": "dev_one",
          "timestamp": "2025-09-22"
        },
        {
          "action": "STATUS_CHANGED",
          "from": "OPEN",
          "to": "IN_PROGRESS",
          "by": "dev_one",
          "timestamp": "2025-09-22"
        },
        {
          "action": "DE-ASSIGNED",
          "by": "dev_one",
          "timestamp": "2025-09-23"
        }
      ],
      "comments": []
    }
  ]
}
```

```
}  
}
```

Advanced Functions

Vizualizarea notificărilor

Disponibilă pentru: Developer

Descriere: Afișează toate notificările primite de developer.

Comanda: viewNotifications

Notificările sunt afișate în ordinea primirii (**cele mai vechi primele**).

După vizualizare, notificările sunt șterse din sistem.

Tip! Pentru detalii despre momentul și conținutul notificărilor, vezi secțiunea [Notificări](#).

Exemplu Input:

```
{  
  "command": "viewNotifications",  
  "username": "user_name",  
  "timestamp": "2025-09-25"  
}
```

Exemplu Output:

```
{  
  "command": "viewNotifications",  
  "username": "user_name",  
  "timestamp": "2025-09-25",  
  "notifications": [  
    "<Notification 1>",  
    "<Notification 2>",  
    "<Notification 3>"  
  ]  
}
```

Cautarea avansată

Disponibilă pentru: Developer, Manager

Descriere: Permite căutarea avansată a tichetelor (sau a developerilor, în cazul managerilor) pe baza unor criterii de filtrare.

Comanda: search

Restricții

1. Toate filtrele vor fi valide pentru rolul utilizatorului.
2. Dacă un filtru nu este specificat → nu se aplică filtrare pe acel câmp.
3. Dacă nu este specificat niciun filtru → comanda este echivalentă cu viewTickets.
4. Dacă nu există rezultate → se returnează o listă goală

Rezultatele sunt afișate după createdAt (cele mai vechi primele), apoi după id în cazul tichetelor și lexicografic după username în cazul developerilor.

Filtre Developer

- Poate căuta: **doar tichete cu status OPEN** din milestone-urile la care este repartizat
- Filtre disponibile:
 - businessPriority
 - type
 - createdAt
 - createdBefore
 - createdAfter
 - availableForAssignment

Filtre manager

- Poate căuta: **toate tichetele din sistem și developerii din subordine**
- Filtre disponibile:
 - toate filtrele Developer
 - keywords
 - expertiseArea
 - seniority
 - performanceScoreAbove
 - performanceScoreBelow

Atenție! **keywords** este case-insensitive și caută potrivirea parțială sau completă a cel puțin un cuvânt în **description** sau **title**.

În output, **matchingWords** arată cuvintele cheie găsite ca potrivire pentru keywords în ordine lexicografică.

Atenție! Dacă **availableForAssignment** = **true**, comanda returnează doar tichetele ce pot fi repartizate developerului curent (în funcție de senioritate, expertiză și milestone).

Atenție! Scorul de performanță al unui developer utilizat în procesul de search este ultimul generat de comanda **generatePerformanceReport**. (0 dacă încă nu a fost dată comanda **generatePerformanceReport**).

Tip! Implementarea căutării trebuie să urmeze **Open-Closed Principle**, permițând adăugarea de noi strategii de filtrare fără modificarea codului existent.

Exemplu Input – Developer:

```
{
  "command": "search",
  "username": "dev_one",
  "timestamp": "2025-09-26",
  "filters": {
    "searchType": "TICKET",
    "businessPriority": "HIGH",
    "type": "BUG",
    "createdAfter": "2025-09-01",
    "availableForAssignment": true
  }
}
```

Exemplu Output – Developer:

```
{
  "command": "search",
  "username": "dev_one",
  "timestamp": "2025-09-26",
  "searchType": "TICKET",
  "results": [
    {
      "id": 0,
      "type": "BUG",
      "title": "Crash on settings page",
      "businessPriority": "HIGH",
      "status": "OPEN",
      "createdAt": "2025-09-10",
      "solvedAt": "",
      "reportedBy": "cleopatra"
    },
    {
      "id": 3,
      "type": "BUG",
      "title": "Data sync failure",
      "businessPriority": "HIGH",
      "status": "OPEN",
      "createdAt": "2025-09-15",
      "solvedAt": "",
      "reportedBy": ""
    }
  ]
}
```

Exemplu Input – Manager (tickets):

```
{
  "command": "search",
  "username": "gabriel_manager",
  "timestamp": "2025-09-26",
  "filters": {
    "searchType": "TICKET",
    "keywords": ["pay", "bug"],
    "businessPriority": "HIGH",
    "type": "BUG",
    "createdAfter": "2025-09-01"
  }
}
```

Exemplu Output – Manager (tickets):

```
{
  "command": "search",
```

```
"username": "gabriel_manager",
"timestamp": "2025-09-26",
"searchType": "TICKET",
"results": [
  {
    "id": 5,
    "type": "BUG",
    "title": "Payment gateway error",
    "businessPriority": "HIGH",
    "status": "OPEN",
    "createdAt": "2025-09-12",
    "solvedAt": "",
    "reportedBy": "user456",
    "matchingWords": ["payment"]
  }
]
```

Exemplu Input – Manager (developers):

```
{
  "command": "search",
  "username": "gabriel_manager",
  "timestamp": "2025-09-26",
  "filters": {
    "searchType": "DEVELOPER",
    "expertiseArea": "BACKEND",
    "seniority": "MID",
    "performanceScoreAbove": number
  }
}
```

Exemplu Output – Manager (developers):

```
{
  "command": "search",
  "username": "gabriel_manager",
  "timestamp": "2025-09-26",
  "searchType": "DEVELOPER",
  "results": [
    {
      "username": "dev_two",
      "expertiseArea": "BACKEND",
      "seniority": "MID",
      "performanceScore": number,
      "hireDate": "2024-05-10"
    }
  ]
}
```

Reports

Raport de performanță

Disponibilă pentru: Manager

Descriere: Generează raportul lunar de performanță pentru toți developerii din subordinea unui manager. În raport sunt incluse toate tichetele rezolvate (status **CLOSED**) din luna anterioară.

Comanda: generatePerformanceReport.

Luna anterioară folosită pentru raport este luna anterioară **timestamp**-ului comenzii.

Developerii vor fi afișați în ordine lexicografică după username.

Un developer fără tichete CLOSED va avea scorul de performanță 0.

Atenție!

averageResolutionTime este media de rezolvare a tuturor tichetelor închise ale unui developer.

averageResolutionTime = assignedAt - solvedAt + 1

Formula:

juniorPerformance = max(0, 0.5 * closedTickets - ticketDiversityFactor) + seniorityBonus

Detalii calcul:

averageResolvedTicketType = (bugTickets + featureTickets + uiTickets) / 3

standardDeviation = sqrt(((bugTickets - avg)^2 + (featureTickets - avg)^2 + (uiTickets - avg)^2) / 3)

```
ticketDiversityFactor = standardDeviation / averageResolvedTicketType
```

Formula:

```
midPerformance = max(0, 0.5 * closedTickets + 0.7 * highPriorityTickets - 0.3 * averageResolutionTime) + seniorityBonus
```

Detalii calcul:

- closedTickets = numărul total de tichete rezolvate
- highPriorityTickets = tichete rezolvate cu prioritate HIGH sau CRITICAL
- averageResolutionTime = media timpilor de rezolvare (zile)

Formula:

```
seniorPerformance = max(0, 0.5 * closedTickets + 1.0 * highPriorityTickets - 0.5 * averageResolutionTime) + seniorityBonus
```

Detalii calcul:

- closedTickets = numărul total de tichete rezolvate
- highPriorityTickets = tichete rezolvate cu prioritate HIGH sau CRITICAL
- averageResolutionTime = media timpilor de rezolvare (zile)
- Impactul tichetelor cu prioritate ridicată și timpul de rezolvare contează mai mult
- Formula penalizează timpi mari de rezolvare și răsplătește rezolvarea tichetelor importante

- JUNIOR** → +5
- MID** → +15
- SENIOR** → +30

Puteți folosi următoarele metode pentru calculul numeric:

```
public static double averageResolvedTicketType(int bug, int feature, int ui) {
    return (bug + feature + ui) / 3.0;
}

public static double standardDeviation(int bug, int feature, int ui) {
    double mean = averageResolvedTicketType(bug, feature, ui);
    double variance = (Math.pow(bug - mean, 2) + Math.pow(feature - mean, 2) + Math.pow(ui - mean, 2)) / 3.0;
    return Math.sqrt(variance);
}

public static double ticketDiversityFactor(int bug, int feature, int ui) {
    double mean = averageResolvedTicketType(bug, feature, ui);

    // dacă nu există tichete, diversitatea este 0
    if (mean == 0.0) {
        return 0.0;
    }

    double std = standardDeviation(bug, feature, ui);
    return std / mean;
}
```

Raportul este calculat doar pentru developerii din echipa manager-ului.

Exemplu Input:

```
{
  "command": "generatePerformanceReport",
  "username": "gabriel_manager",
  "timestamp": "2025-09-01"
}
```

Exemplu Output:

```
{
  "command": "generatePerformanceReport",
  "username": "gabriel_manager",
  "timestamp": "2025-09-01",
  "report": [
    {

```

```
{
  "username": "dev_one",
  "closedTickets": 5,
  "averageResolutionTime": 4.2,
  "performanceScore": 88.56,
  "seniority": "SENIOR"
},
{
  "username": "dev_two",
  "closedTickets": 3,
  "averageResolutionTime": 5.0,
  "performanceScore": 75.00,
  "seniority": "MID"
}
]
```

Raport de risc

Disponibilă pentru: Manager

Descriere: Generează un raport pe baza metricii **ticket risk**.

Comanda: **generateTicketRiskReport**

Raportul este creat pe baza **tuturor** tichetelor **OPEN** și **IN_PROGRESS** din sistem, indiferent de managerul care a generat raportul.

Tip! Pentru mai multe detalii despre metrice, vizitați [Metrici de stabilitate](#).

Calculare:

Formula: **frequency × severityFactor**

Valoare maximă: 12

Exemplu de calcul:

```
frequency = ALWAYS (4)
severityFactor = SEVERE (3)

Risc brut = 4 × 3 = 12
Risc final = (12 × 100) / 12 = 100.00
```

Formula: **businessValue + customerDemand**

Valoare maximă: 20

Exemplu de calcul:

```
businessValue = XL (10)
customerDemand = HIGH (6)

Risc brut = 10 + 6 = 16
Risc final = (16 × 100) / 20 = 80.00
```

Formula: **(11 – usabilityScore) × businessValue**

Valoare maximă: 100

Exemplu de calcul:

```
usabilityScore = 2
businessValue = XL (10)

Risc brut = (11 – 2) × 10 = 90
Risc final = (90 × 100) / 100 = 90.00
```

În urma raportului, fiecare categorie va avea un **calificativ**:

Interval Risc	Calificativ
0 – 24	NEGLIGIBLE
25 – 49	MODERATE
50 – 74	SIGNIFICANT
75 – 100	MAJOR

Exemplu Input:

```
{
  "command": "generateTicketRiskReport",
  "username": "gabriel_manager",
}
```

```
"timestamp": "2025-09-30"
}
```

Exemplu Output:

```
{
  "command": "generateTicketRiskReport",
  "username": "gabriel_manager",
  "timestamp": "2025-09-30",
  "report": {
    "totalTickets": 20,
    "ticketsByType": {
      "BUG": 10,
      "FEATURE_REQUEST": 6,
      "UI_FEEDBACK": 4
    },
    "ticketsByPriority": {
      "LOW": 5,
      "MEDIUM": 7,
      "HIGH": 6,
      "CRITICAL": 2
    },
    "riskByType": {
      "BUG": "NEGLIGIBLE",
      "FEATURE_REQUEST": "MODERATE",
      "UI_FEEDBACK": "SIGNIFICANT"
    }
  }
}
```

Raport de eficiență a rezolvării

Disponibilă pentru: Manager

Descriere: Evaluează cât de eficientă este echipa în a rezolva tichete. Se vor considera doar tichete în stările **RESOLVED** și **CLOSED**.

Comanda: generateResolutionEfficiencyReport

Raportul este creat pe baza **tuturor tichetelor CLOSED** si **RESOLVED** din sistem, indiferent de managerul care a generat raportul.

Tip! Pentru mai multe detalii despre metrice, vizitați [Metrice de stabilitate](#).

Calculare:

Formula:

$(\text{bugFrequency} + \text{severityFactor}) \times 10 / \text{daysToResolve}$

Valoare maximă: 70

Exemplu de calcul:

```
frequency = FREQUENT (3)
severityFactor = MODERATE (2)
daysToResolve = 2

Scor eficiență = (3 + 2) × 10 / 2 = 25
Eficiență finală = (25 × 100) / 70 = 35.71
```

Formula:

$(\text{businessValue} + \text{customerDemand}) / \text{daysToResolve}$

Valoare maximă: 20

Exemplu de calcul:

```
businessValue = L (6)
customerDemand = HIGH (6)
daysToResolve = 2

Scor eficiență = (6 + 6) / 2 = 6.00
Eficiență finală = (6 × 100) / 20 = 30.00
```

Formula:

$(\text{usabilityScore} + \text{businessValue}) / \text{daysToResolve}$

Valoare maximă: 20

Exemplu de calcul:

```
usabilityScore = 9
businessValue = XL (10)
```

```
Scor eficiență = (9 + 10) / 2 = 9.5
Eficiență finală = (9.5 × 100) / 20 = 47.50
```

Atenție! `daysToResolve` = numărul de zile dintre `assignedAt` și data la care tichetul a fost marcat ultima dată ca **RESOLVED** sau **CLOSED**.

Exemplu Input:

```
{
  "command": "generateResolutionEfficiencyReport",
  "username": "gabriel_manager",
  "timestamp": "2025-09-30"
}
```

Exemplu Output:

```
{
  "command": "generateResolutionEfficiencyReport",
  "username": "gabriel_manager",
  "timestamp": "2025-09-30",
  "report": {
    "totalTickets": 20,
    "ticketsByType": {
      "BUG": 10,
      "FEATURE_REQUEST": 6,
      "UI_FEEDBACK": 4
    },
    "ticketsByPriority": {
      "LOW": 5,
      "MEDIUM": 7,
      "HIGH": 6,
      "CRITICAL": 2
    },
    "efficiencyByType": {
      "BUG": 65.50,
      "FEATURE_REQUEST": 55.30,
      "UI_FEEDBACK": 40.25
    }
  }
}
```

Raport de impact asupra clienților

Disponibilă pentru: Manager

Descriere: Estimează **impactul asupra clienților** al tichetelor.

Comanda: `generateCustomerImpactReport`

Raportul este creat pe baza **tuturor tichetelor** aflate în stările **OPEN** și **IN_PROGRESS**, indiferent de managerul care a generat raportul.

Tip! Pentru mai multe detalii despre metrice, vizitați [Metrice de stabilitate](#).

Calculare:

Formula:

$frequency \times businessPriority \times severityFactor$

Valoare maximă: 48

Exemplu de calcul:

```
frequency = ALWAYS (4)
businessPriority = CRITICAL (4)
severityFactor = SEVERE (3)

Impact brut = 4 × 4 × 3 = 48
Impact final = (48 × 100) / 48 = 100.00
```

Formula:

$businessValue \times customerDemand$

Valoare maximă: 100

Exemplu de calcul:

```
businessValue = M (3)
customerDemand = HIGH (6)

Impact brut = 3 × 6 = 18
Impact final = (18 × 100) / 100 = 18.00
```


Formula:
$$\text{businessValue} \times \text{usabilityScore}$$
Valoare maximă: 100**Exemplu de calcul:**

```
businessValue = L (6)
usabilityScore = 9

Impact brut = 6 × 9 = 54
Impact final = (54 × 100) / 100 = 54.00
```

Exemplu Input:

```
{
  "command": "generateCustomerImpactReport",
  "username": "gabriel_manager",
  "timestamp": "2025-09-30"
}
```

Exemplu Output:

```
{
  "command": "generateCustomerImpactReport",
  "username": "gabriel_manager",
  "timestamp": "2025-09-30",
  "report": {
    "totalTickets": 20,
    "ticketsByType": {
      "BUG": 10,
      "FEATURE_REQUEST": 6,
      "UI_FEEDBACK": 4
    },
    "ticketsByPriority": {
      "LOW": 5,
      "MEDIUM": 7,
      "HIGH": 6,
      "CRITICAL": 2
    },
    "customerImpactByType": {
      "BUG": 75.55,
      "FEATURE_REQUEST": 60.05,
      "UI_FEEDBACK": 45.02
    }
  }
}
```

Raport de stabilitate a aplicației

Disponibilă pentru: Manager**Descriere:** Generează un raport de stabilitate al aplicației, pe baza metricii **Ticket Risk** și a metricii **Customer Impact**.**Comanda:** appStabilityReportRaportul combină **Ticket Risk** și **Customer Impact** pentru a determina stabilitatea aplicației la un moment dat.

Raportul este creat pe baza tuturor tichetelor din sistem, indiferent de managerul care a generat raportul.

Restricții

1. Dacă nu există tichete **OPEN** sau **IN_PROGRESS**, aplicația este considerată **STABLE**.
2. Dacă toate categoriile de risc sunt **NEGLIGIBLE** și impactul pentru clienți este sub 50 pentru toate tipurile de tichet, aplicația este considerată **STABLE**.
3. Dacă există cel puțin o categorie de risc **SIGNIFICANT**, aplicația este considerată **UNSTABLE**.
4. În celelalte cazuri, aplicația este considerată **PARTIALLY STABLE**.
5. Dacă aplicația este **STABLE**, aplicația își încheie execuția și nu mai sunt procesate alte comenzi.

Exemplu Input:

```
{
  "command": "appStabilityReport",
  "username": "gabriel_manager",
  "timestamp": "2025-09-30"
}
```

Exemplu Output: Exemplu Output:

```
{
  "command": "appStabilityReport",
  "username": "gabriel_manager",
  "timestamp": "2025-09-30",
  "report": {
    "totalOpenTickets": 15,
    "openTicketsByType": {
      "BUG": 8,
      "FEATURE_REQUEST": 5,
      "UI_FEEDBACK": 2
    },
    "openTicketsByPriority": {
      "LOW": 3,
      "MEDIUM": 6,
      "HIGH": 4,
      "CRITICAL": 2
    },
    "riskByType": {
      "BUG": "SIGNIFICANT",
      "FEATURE_REQUEST": "MODERATE",
      "UI_FEEDBACK": "NEGLIGIBLE"
    },
    "impactByType": {
      "BUG": 87.45,
      "FEATURE_REQUEST": 42.30,
      "UI_FEEDBACK": 5.75
    },
    "appStability": "UNSTABLE"
  }
}
```

Scheletul de cod

Scheletul de cod conține:

- Clasa **App** care este punctul de intrare pentru logica de rezolvare a temei.

În cadrul acestei teme, **citirea datelor de intrare nu mai este implementată în schelet**.

Recomandăm să folosiți **librăria Jackson** la capacitate maximă, profitând de **anotările sale** (**@JsonSubTypes**, **@JsonProperty**, **@JsonIgnore**, **@JsonCreator**, etc.) pentru a simplifica procesul de mapare al obiectelor.

Pentru mai multe detalii puteți vizita Jackson Annotation Examples (Baeldung) [https://www.baeldung.com/jackson-annotations] și Jackson Object Mapper tutorial (Baeldung) [https://www.baeldung.com/jackson-object-mapper-tutorial].

- Implementarea voastră va începe din metoda **App.run**, unde veți adăuga în **ArrayNode output** pe parcursul execuției toate output-urile comezilor date. **De asemenea, aveți un mic exemplu în schelet despre cum puteți face asta.**
- Aveți în fișierul **pom.xml** toate dependențele necesare pentru rularea temei, mai exact bibliotecile Jackson și câteva dependențe folosite pentru testare.

Output-ul **nu trebuie** formatat ca în ref-uri, fiindcă se verifică conținutul obiectelor și array-urilor JSON, nu textul efectiv.

Totuși vă recomandăm să utilizați **PrettyPrinter** dacă folosiți Jackson pentru că vă ajută la depanare (Documentație PrettyPrinter [https://fasterxml.github.io/jackson-databind/javadoc/2.7/com/fasterxml/jackson/databind/ObjectMapper.html#writerWithDefaultPrettyPrinter()]).

Totodată, pentru a înțelege cum se poate realiza **scrierea în fișierele JSON de output**, vă sugerăm să consultați JSON Array [https://atacomsian.com/blog/jackson-create-json-array].

Dacă aveți probleme cu proiectul vostru **vă recomandăm** următoarele soluții:

- Să închideți proiectul din IntelliJ, să ștergeți din file explorer folderele .idea și target și să deschideți din nou folderul proiectului cu IntelliJ (File → open → selectați folderul în care se află pom.xml). Acest pas vă **regenerează configurația IntelliJ-ului**.
- Să apăsați pe iconița "M" din dreapta în IntelliJ, după care să apăsați pe item-ul din listă, apoi pe lifecycle și apoi dublu click pe install. Acest pas vă **reinstalează dependențele proiectului**.
- Să apăsați pe File → Invalidate cache și să bifați toate opțiunile. Proiectul o să se reîncarce **ștergând toate configurațiile și fișierele cached**.

Execuția temei

Pentru a rula checker-ul intrați în fișierul **"src/test/java/TestRunner"** și rulați clasa.

O să vi se deschidă un panou în IntelliJ de unde puteți vedea **statusul tuturor testelor**. De asemenea, vi se va afișa și **eroarea sau diferența rezultatului** pe care îl aveți pentru **fiecare test** în parte.

Pentru ca checker-ul să funcționeze trebuie să deschideți tema din IntelliJ la calea unde se află fișierul **"pom.xml"**.

Recomandări

- Tema a fost concepută pentru a vă testa cunoștințele dobândite în urma parcurgerii laboratoarelor 1-9, aceasta putând fi rezolvată doar cu noțiunile învățate din acele laboratoare.
- Tema fiind o specificație destul de stufoasă recomandăm citirea de cel puțin două ori a enunțului înainte de începerea implementării.

Verificați periodic această pagină, deoarece scheletul/cerința pot suferi modificări în urma unor erori din partea noastră.

Utilizarea LLM-urilor (ChatGPT, Gemini, Claude etc.)

Puteți folosi modele de limbaj (LLM-uri) pentru **asistență** în realizarea temei doar ca **instrument suplimentar**, de exemplu pentru clarificări, explicații sau verificarea ideilor.

Totuși, **recomandarea noastră** este să încercați să **rezolvați tema pe cont propriu**, fără a depinde de astfel de instrumente. Scopul exercițiului este să **învățați** și să vă **dezvoltați abilitățile**, iar progresul real vine doar prin înțelegere și practică personală.

Puteți folosi LLM-uri pentru:

- explicații teoretice.
- exemple suplimentare.
- verificarea codului sau a ideilor.
- clarificarea unor concepte.

NU este acceptat să:

- trimiteți soluții generate integral de un LLM.
- folosiți cod sau text pe care nu îl înțelegeți.
- copiați fără să puteți explica ce ați făcut în cod.
- generați README-ul folosind un LLM.

Evaluare

Punctajul constă din:

- 80p implementare - trecerea testelor
 - 10p coding style (vezi checkstyle)
 - 5p README.md clar, concis, explicații axate pe design (flow, interacțiuni)
 - 5p folosire git pentru versionarea temei
- Pe pagina [Indicații pentru temă](#) găsiți indicații despre scrierea README-ului și **baremul folosit la corectarea temei**. Vă încurajăm să treceți prin el cel puțin odată.
- Vă recomandăm să folosiți **README.md**, deoarece vă puteți formata conținutul mai bine.

Depunctările pentru temă sunt acoperite în barem, totuși pot apărea depunctări **care nu se află în imaginile furnizate**.

Bonusuri: La evaluare, putem oferi bonusuri pentru design foarte bun, cod bine documentat dar și pentru diverse elemente suplimentare.

- **Temele vor fi testate împotriva plagiatului.** Orice tentativă de copiere va duce la **anularea punctajului** de pe parcursul semestrului și **repetarea materiei** atât pentru sursă(e) cât și pentru destinație(ii), fără excepție.
- Puteți folosi LLM-uri (ChatGPT, Gemini etc.) doar pentru clarificări sau fragmente mici de cod, însă **menționați în README** unde și cum au fost folosite pentru a evita orice scenariu de copiat.

Git

Folosirea git pentru versionare va fi verificată din folderul .git pe care trebuie să îl includeți în arhiva temei. Punctajul se va acorda dacă ați făcut **minim 3 commit-uri relevante și cu mesaj sugestiv**. **NU** este permis să aveți repository-urile de git publice până la deadline-ul hard.

Pentru a primi punctajul pentru **Git**, după ce ați terminat tema și ați făcut toate commit-urile, adăugați directorul .git în arhiva temei.

Pentru folosirea tool-ului **Git** vă punem la dispoziție un [tutorial](#) actualizat și amplu și aveți de asemenea și un [tutorial](#) despre comenzile pe care puteți să le dați din IntelliJ la acest.

- După ce clonați repo-ul de pe GitHub, vă rugăm să vă faceți un repository propriu **privat** cu conținutul temei sau să faceți un *fork* privat. Dacă nu folosiți un repository sau fork propriu **nu o să puteți** să urcați schimbările din Git în GitHub, deoarece vă aflați în rădăcina repository-ului echipei de POO și **aveți riscul să vă expuneți soluția**.
- Vă rugăm să vă generați un **.gitignore** cu template-ul **Maven** dacă nu folosiți gitignore-ul inclus în temă. Acesta vă ajută să definiți fișiere care să nu fie luate în considerare la **diff check** când rulați **git status/git commit** sau alte comenzi de git.

Checkstyle

Unul din obiectivele temei este învățarea respectării code-style-ului limbajului pe care îl folosiți. Aceste convenții (de exemplu cum numiți fișierele, clasele, variabilele, cum indentați) sunt verificate pentru temă de către tool-ul checkstyle [<https://checkstyle.sourceforge.io/>] (care se află în pom.xml ca dependență).

Pe pagina de [Recomandări cod](#) găsiți câteva exemple de coding style.

Dacă numărul de erori depistate de checkstyle depășește 30, atunci punctele pentru coding-style nu vor fi acordate. Dacă punctajul este negativ, *acesta se trunchiază la 0*.

Exemple:

- punctaj_total = 100 și nr_erori = 200 ⇒ nota_finala = 90
- punctaj_total = 100 și nr_erori = 29 ⇒ nota_finala = 100
- punctaj_total = 80 și nr_erori = 30 ⇒ nota_finala = 80
- punctaj_total = 80 și nr_erori = 31 ⇒ nota_finala = 70

Teste

1. in_01_test_report.json

- 2. in_02_test_milestone.json
- 3. in_03_test_milestone_edge_case.json
- 4. in_04_test_assign.json
- 5. in_05_test_assign_edge_case.json
- 6. in_06_test_comment.json
- 7. in_07_test_comment_edge_case.json
- 8. in_08_test_status_change.json
- 9. in_09_test_status_undo_change.json
- 10. in_10_test_status_edge_case.json
- 11. in_11_test_search.json
- 12. in_12_test_notifications.json
- 13. in_13_test_metrics_customer_impact.json
- 14. in_14_test_metrics_ticket_risk.json
- 15. in_15_test_metrics_efficiency.json
- 16. in_16_test_stability.json
- 17. in_17_test_performance.json
- 18. in_18_test_complex.json
- 19. in_19_test_complex_edge_case.json

Upload temă

Arhiva o veți urca pe Code Devmind, unde sunt și informații despre structura ei.

Checker-ul din cloud **nu a fost publicat încă!** Vă vom comunica pe Teams când acesta devine disponibil.

FAQ

Q: Pot folosi biblioteca "X"?

A: Dacă doriți să folosiți o anumită bibliotecă vă recomandăm să întrebați pe forum, ca apoi să o validăm și să o includem și în pom-ul de pe Code Devmind.

Q: Pot folosi GSON în loc de Jackson?

A: Sigur, contează doar să aveți output-ul corect și să respectați convențiile de coding style și modularitate.

Q: Am descoperit edge case-ul "Y", trebuie să îl tratez?

A: Nu. Toate datele necesare pentru soluționarea temei vă sunt date în cerință. Dacă totuși am omis ceva ne puteți contacta pe forum.

Q: Pot folosi clase de tip "Enum" pentru constante?

A: Da.

Q: Ce JDK recomandați?

A: 25

Q: Pot să fac în orice ordine testele?

A: Depinde. Testele au fost concepute să fie cât mai decuplate și să testeze câte o funcționalitate în întregime. Cu toate acestea, vă recomandăm să implementați mai întâi testele 01, 02, 03 deoarece reprezintă funcționalitățile de bază pentru rezolvarea următoarelor teste mai complexe.

Resurse și linkuri utile

- Schelet de cod [<https://github.com/oop-pub/schelet-tema2-2025>]
- Forum [<https://curs.upb.ro/2025/mod/forum/view.php?id=43092>]
- [Indicații pentru temă](#)
- [Recomandări coding style & javadoc](#)
- [Tutorial Git](#)

poo-ca-cd/teme/2025/b73f56dc-17a1-42ac-bd7e-d57f3caaf9fd/tema-2.txt · Last modified: 2026/01/04 09:00 by valentin.carauleanu